PETWELL

PetWell Frontend

Developer & Onboarding Documentation

A first-principles guide to the Next.js frontend — the App Router structure, rendering model, authentication, the data layer, server actions, and every component and feature, with the reasoning behind each design decision.

Next.js 16	React 19	TypeScript	Tailwind v4	next-auth v5

Frontend Documentation · July 2026

Pet Health Passport & Appointment Management System

Table of Contents

1. Introduction
– What is the PetWell frontend?
– Who uses it
– How to read this document

2. System Architecture
– The rendering model
– The life of a page render
– How the frontend talks to the backend

3. Technology Stack & Key Concepts
– The stack
– App Router building blocks
– Server Components vs. Client Components
– Server Actions

4. Project Structure
– Anatomy of the src tree
– The three layers: app / components / lib

5. Application Bootstrap & Root Layout
– app/layout.tsx
– Fonts, metadata & the Toaster
– globals.css

6. Routing & Route Groups
– Route groups and areas
– Nested layouts
– Dynamic segments
– middleware.ts

7. Authentication & Session
– next-auth Credentials provider
– jwt & session callbacks
– Session helpers
– Route protection

8. The Data Layer
– apiFetch
– authFetch
– ApiError
– The API clients
– Response types

9. Server Actions & Validation
– The 'use server' boundary
– The ActionResult contract
– Zod schemas
– Cache revalidation

10. Component Library
– UI primitives (Base-UI)

– Shared components
– Layout components
– Feature components

11. Feature Walkthroughs

– Authentication
– Pets
– Health passport
– Clinics & booking
– Appointments
– Profile

12. Styling & the Design System

– Tailwind v4 setup
– Design tokens & the .pw-* layer

13. Cross-Cutting Concerns

– Data fetching pattern
– Loading & pending states
– Error handling & empty states
– Images
– Accessibility

Appendices

– A — Route map
– B — Server Actions reference
– C — API client reference
– D — Glossary
– E — Running the frontend

1 Introduction

This document explains the entire PetWell frontend — a web application built with `Next.js 16` (App Router), React 19 and TypeScript — in enough depth that a developer who has never seen the project can understand not just what the code does, but why it is structured the way it is. It is the companion to the backend documentation: where that guide covers the NestJS REST API, this one covers the client that consumes it.

1.1 What is the PetWell frontend?

PetWell is a Pet Health Passport & Appointment Management System. The frontend is the part the user actually sees and touches: the pages, forms, dashboards and navigation. It renders almost everything on the server (React Server Components), calls the NestJS API for data, and uses Server Actions to perform writes. It never talks to the database directly — every piece of data flows through the backend's HTTP API.

> **The frontend in one sentence** Server Components fetch data through a typed API client, Server Actions perform writes and revalidate the cache, and next-auth carries the user's JWT so the backend knows who is asking.

1.2 Who uses it

Role	Who	What they do in the UI
OWNER	Pet owner	Manage pets, view the portable health passport, search clinics, book & cancel appointments, edit their profile.
CLINIC_STAFF	Vet / clinic employee	Look up any pet, write & update health records, manage their clinic's appointment queue, edit the clinic profile.

The two roles see two different areas of the app — `/owner/*` and `/staff/*` — but both are built from the same components and the same design language.

1.3 How to read this document

If you are brand new, read Chapter 2 (architecture) and Chapter 7 (authentication) first. Once you understand how a page is rendered and how the user's identity travels with each request, every feature in Chapter 11 follows the same shape: a Server Component fetches data, a form collects input, a Server Action validates it and calls the API, and the cache revalidates.

2 System Architecture

Before opening a single file it helps to see the shape of the whole client. PetWell's frontend follows the Next.js App Router model: the app is a tree of nested layouts and pages, most of which run on the server, with small islands of interactivity (Client Components) where the user types or clicks.

2.1 The rendering model

Three distinct execution contexts coexist. Understanding which code runs where is the single most important idea in the codebase.

Context	Runs on	Responsibility	Marker
Server Component	Server (per request)	Fetch data, compose layout, render HTML. The default for every file under app/.	`(no directive)`
Client Component	Browser	Interactivity: state, effects, event handlers, forms.	`'use client'`
Server Action	Server (on submit)	Mutations: validate input, call the API, revalidate cache.	`'use server'`

2.2 The life of a page render

A request for an authenticated page travels through a fixed sequence. Take `/owner/pets` as the example:

- **middleware.ts** runs at the edge first — it checks the session cookie and the user's role, and redirects to /login or the other area if the visit isn't allowed.
- The matching **layout** (app/owner/layout.tsx) runs requireRole('OWNER'), then renders the AppShell (sidebar + header) around the page.
- The **page** (a Server Component) calls PetApi.listMine(), which attaches the user's JWT and fetches from the NestJS API.
- React renders the resulting HTML on the server and streams it to the browser.
- Any **Client Components** inside (e.g. an 'Add pet' form) hydrate and become interactive.

> **Why server-first** Rendering on the server means the JWT and API calls stay off the client, pages arrive as ready HTML, and the browser bundle stays small — only the interactive islands ship JavaScript.

2.3 How the frontend talks to the backend

There is exactly one door between the frontend and the outside world: the data layer in `lib/api/`. Every call goes through `apiFetch` (public requests) or `authFetch` (authenticated requests, which attach the JWT). No component fetches directly. This keeps base-URL handling, JSON encoding, error mapping and auth in one place — covered in full in Chapter 8.

3 Technology Stack & Key Concepts

3.1 The stack

Concern	Choice	Why
Framework	Next.js 16 (App Router)	Server Components, Server Actions, file-system routing, middleware.
UI library	React 19	Latest concurrent features; useTransition for pending UI.

Concern	Choice	Why
Language	TypeScript 5	End-to-end types from API response shapes to component props.
Auth	next-auth v5 (beta)	Credentials provider wrapping the backend login; JWT session strategy.
Forms	react-hook-form + zod	Typed, schema-validated forms via @hookform/resolvers.
Styling	Tailwind CSS v4 + Base-UI	Utility CSS with a small set of headless primitives (shadcn-style).
Feedback	sonner	Toast notifications for action results.
Dates	date-fns + react-day-picker	Formatting and the calendar/date-picker.

3.2 App Router building blocks

- **page.tsx** — the UI for a route. A Server Component by default.
- **layout.tsx** — shared shell that wraps a page and all nested routes; persists across navigations.
- **Route groups** — folders in (parentheses) that group routes without adding a URL segment, e.g. (auth).
- **Dynamic segments** — folders in [brackets], e.g. [petId], whose value arrives as a param.
- **middleware.ts** — code that runs before a request is completed, used here for auth gating.

3.3 Server Components vs. Client Components

Every file under `app/` is a Server Component unless it opens with `'use client'`. Server Components can be `async` and await data directly; they cannot use state or browser APIs. Client Components can use hooks and handle events but run in the browser. PetWell keeps Client Components small and pushes data fetching up into Server Components.

3.4 Server Actions

A Server Action is an `async` function marked `'use server'` that a form or client can call directly; Next.js turns it into a secure server endpoint. PetWell uses actions for every write — they validate with zod, call the API client, and revalidate affected pages. They always return a typed `ActionResult` so the UI can show success or the exact error.

4 Project Structure

4.1 Anatomy of the src tree

```
src/
■■ app/                      # App Router: routes, layouts, pages
■  ■■ layout.tsx             # root layout (fonts, Toaster, metadata)
■  ■■ globals.css            # Tailwind + design tokens + .pw-* classes
■  ■■ page.tsx               # public landing page
■  ■■ (auth)/                # route group: /login, /register
■  ■■ owner/                 # Pet Owner area (layout + pages)
■  ■■ staff/                 # Clinic Staff area (layout + pages)
■  ■■ api/auth/[...nextauth] # next-auth route handler
■■ components/
■  ■■ ui/                    # Base-UI primitives (button, input, …)
■  ■■ shared/                # Logo, PhotoPlaceholder, StatusBadge
■  ■■ layout/                # AppShell, ProfileMenu, PublicHeader
■  ■■ features/              # auth, pet, appointment, clinic, …
■■ lib/
■  ■■ api/                   # apiFetch, authFetch, typed API clients
■  ■■ actions/               # Server Actions (writes)
■  ■■ schemas/               # zod schemas + inferred input types
■  ■■ auth.ts                # next-auth config
■  ■■ session.ts             # getSessionUser / requireUser / requireRole
■  ■■ utils.ts               # cn() class-name helper
■■ middleware.ts             # route protection
```

4.2 The three layers

The tree divides cleanly into three responsibilities. `app/` is routing and presentation, `components/` is reusable UI, and `lib/` is everything non-visual: data access, actions, validation and auth. A new feature almost always touches all three — a page in app/, a form in components/features/, and a schema + action + API method in lib/.

5 Application Bootstrap & Root Layout

5.1 app/layout.tsx

The root layout wraps every page in the app. It sets the `<html>` and `<body>`, loads the Geist fonts as CSS variables, defines the default page-title template, and mounts the global `<Toaster/>` once so any Server Action can raise a toast.

```
export const metadata: Metadata = {
  title: { template: '%s | PetWell',
           default: 'PetWell — Pet Health Passport' },
};

export default function RootLayout({ children }) {
  return (
    <html lang='en' className={`${geistSans.variable} …`}>
      <body className='min-h-full flex flex-col bg-muted/30'>
        {children}
        <Toaster richColors position='top-center' />
      </body>
    </html>
  );
}
```

> **Note on fonts** The root layout loads Geist as the technical default, while the product typography (Inter + Playfair Display) is set in globals.css and applied via the .font-display class. See Chapter 12.

5.2 globals.css

The single stylesheet imports Tailwind v4, defines the design tokens (the ivory/charcoal/dusty-blue palette), and declares the `.pw-*` utility layer — `.pw-btn-primary`, `.pw-lift`, `.pw-fade` — used across the product UI. It is documented in full in the standalone Design System page (see Chapter 12).

6 Routing & Route Groups

6.1 Route groups and areas

The app splits into three zones. Public routes live at the root, the two authenticated areas live under `owner/` and `staff/`, and the `(auth)` group holds login/register without adding an /auth segment to the URL.

Path	Area	Layout
/	Public landing	root only
/login, /register	Public auth	(auth)/layout.tsx (PublicHeader)
/owner/*	Pet Owner	owner/layout.tsx (AppShell, requireRole OWNER)
/staff/*	Clinic Staff	staff/layout.tsx (AppShell, requireRole CLINIC_STAFF)

6.2 Nested layouts

Each area layout is an `async` Server Component that first calls `requireRole(...)` (redirecting anyone who shouldn't be there) and then renders the shared `AppShell` with the correct navigation. Every page inside the area is rendered as the AppShell's child, so the sidebar and header are guaranteed and never re-mount between navigations.

```
export default async function OwnerLayout({ children }) {
  const user = await requireRole('OWNER');
  return (
    <AppShell area='owner' userName={`${user.firstName} ${user.lastName}`}>
      {children}
    </AppShell>
  );
}
```

6.3 Dynamic segments

Detail pages use dynamic segments: `owner/pets/[petId]/page.tsx` receives `params` (a Promise in Next 16) and awaits it to read `petId`, then fetches that pet. If the API returns nothing, the page calls `notFound()` to render the 404.

6.4 middleware.ts

Middleware is the first line of route protection, running before any page. It reads the session, derives the role, and enforces the rules: signed-in users are bounced off the auth/landing pages to their dashboard; `/owner/*` and `/staff/*` require the matching role. The `matcher` excludes API routes, static files and assets so it only runs on real pages.

> **Defence in depth** Middleware guards the UI, but it is not the only gate: each area layout re-checks the role, and the backend independently authorises every API call from the JWT. A forged cookie still can't read another user's data.

7 Authentication & Session

Authentication is handled by next-auth v5 configured in `lib/auth.ts`. PetWell does not manage its own passwords on the client — it delegates to the backend and stores the returned JWT inside the next-auth session.

7.1 The Credentials provider

A single Credentials provider defines `authorize`: it validates the login input with the zod `loginSchema`, calls `AuthApi.login`, and on success returns the user object plus the backend's `access_token`. Any failure returns `null`, which next-auth treats as invalid credentials.

```
Credentials({
  async authorize(input) {
    try {
      const data = loginSchema.parse(input);
      const { access_token, user } = await AuthApi.login(data);
      return { ...user, access_token };
    } catch { return null; }
  },
})
```

7.2 jwt & session callbacks

Two callbacks shape what the session carries. The `jwt` callback copies the user fields (id, name, role, clinicId, avatarUrl) and the `access_token` into the token on login, and merges updates when the profile changes. The `session` callback then exposes those on `session.user` so both Server Components and authFetch can read them. The session `maxAge` is set to 86370s — just under the backend's 24-hour token — so the cookie never outlives the JWT it carries.

7.3 Session helpers

Three helpers in `lib/session.ts` standardise access:

Helper	Behaviour
getSessionUser()	Returns the session user or null. Non-redirecting.
requireUser()	Returns the user or redirects to /login.
requireRole(role)	Requires a specific role; bounces to the other area's dashboard on mismatch.

7.4 Route protection recap

Protection is layered: middleware gates the URL, the area layout calls requireRole, and `authFetch` redirects to /login if the token is missing when a data call is made. The backend is the final authority.

8 The Data Layer

All communication with the backend flows through `lib/api/`. This layer is small but central: it owns the base URL, JSON encoding, token attachment, error mapping and the TypeScript response types.

8.1 apiFetch

The primitive. It prefixes the API base URL, attaches a Bearer token when one is passed, JSON-encodes plain-object bodies (leaving FormData alone so uploads keep their multipart boundary), throws a typed `ApiError` on any non-2xx, and tolerates empty or non-JSON responses (upload endpoints return a raw URL string).

```
const API_URL = process.env.API_URL ?? 'http://localhost:8000';

if (!response.ok) {
  let message = response.statusText;
  try {
    const errorBody = await response.json();
    message = Array.isArray(errorBody.message)
      ? errorBody.message.join(', ')
      : (errorBody.message ?? message);
  } catch { /* non-JSON error body */ }
  throw new ApiError(response.status, message);
}
```

8.2 authFetch

A thin wrapper over apiFetch for authenticated calls. It reads the JWT from the next-auth session and attaches it; if there is no session it `redirect()`s to /login. Every owner/staff data call uses authFetch; only login, register and the public clinic browse use the bare apiFetch.

8.3 ApiError

A small Error subclass carrying `statusCode` and `message`. Server Actions catch it to convert an API failure into a friendly `ActionResult`; anything that isn't an ApiError is re-thrown as a genuine bug.

8.4 The API clients

Each resource has a typed client object — `PetApi`, `AppointmentApi`, `ClinicApi`, `HealthRecordApi`, `UserApi`, `AuthApi` — whose methods name intent and return typed data. Components and actions call these, never fetch directly. The full endpoint mapping is Appendix C.

8.5 Response types

Every backend response shape is mirrored in `lib/api/api.type.ts`: `UserResponse`, `Pet`, `Clinic`, `HealthRecord`, `Appointment`, `AppointmentStatusLog`, plus the enums (Role, ClinicType, RecordType, AppointmentStatus). These types flow through the clients into components, so a change to a shape surfaces as a compile error everywhere it matters.

9 Server Actions & Validation

9.1 The 'use server' boundary

Files in `lib/actions/` open with `'use server'`. Their exported functions can be imported into Client Components and called like normal functions, but they execute on the server. This is where every write lives — create, update, cancel, upload.

9.2 The ActionResult contract

Every action returns the same discriminated union, so the calling form can branch on one field:

```
export type ActionResult =
  | { success: true; message?: string }
  | { success: false; message: string;
      errors?: Record<string, unknown>; code: string };
```

The pattern inside each action is uniform: parse with zod → call the API client → catch ApiError and return a failure → on success, revalidate and return `{ success: true }`.

```
export async function createPetAction(input: CreatePetInput) {
  const parsed = createPetSchema.safeParse(input);
  if (!parsed.success)
    return { success: false, message: 'Please check the form fields.',
             code: 'VALIDATION_ERROR' };
  try { await PetApi.create(parsed.data); }
  catch (e) {
    if (e instanceof ApiError)
      return { success: false, message: e.message, code: 'API_ERROR' };
    throw e;
  }
  revalidatePath('/owner/pets');
  revalidatePath('/owner/dashboard');
  return { success: true, message: 'Pet added.' };
}
```

9.3 Zod schemas

Schemas in `lib/schemas/` are the single source of truth for form validation. They are used twice: react-hook-form validates the form live via `standardSchemaResolver`, and the Server Action re-validates on the server with `safeParse` before trusting the input. The inferred `z.infer` types are the input types passed to the API clients.

9.4 Cache revalidation

After a successful write, an action calls `revalidatePath(...)` for every page whose data changed — e.g. creating a pet revalidates both the pet list and the dashboard. Next then re-renders those Server Components on the user's next visit, so the UI reflects the new state without a manual refetch.

10 Component Library_

Components live in four buckets under `components/`. Chapter 12 and the standalone Design System page cover their visual styling; this chapter covers their role in the code.

10.1 UI primitives (components/ui)

Headless Base-UI / shadcn-style primitives: `button`, `input`, `textarea`, `select`, `badge`, `avatar`, `card`, `dialog`, `popover`, `label`, `calendar`, `date-picker`, and the sonner `Toaster`. These are generic and reusable; the product look is layered on via the .pw-* classes.

10.2 Shared components (components/shared)

- **Logo / PawMark** — the brand wordmark and paw glyph.
- **PhotoPlaceholder** — the ivory fallback tile for missing pet/clinic images.
- **StatusBadge** — the canonical appointment-status pill (six colour pairs).

10.3 Layout components (components/layout)

- **AppShell** — the authenticated shell: desktop sidebar, sticky header, mobile bottom-tab bar; takes an area ('owner'|'staff') and the user name.

- **ProfileMenu** — the header avatar dropdown (profile link + logout), with correct ARIA and outside-click close.
- **PublicHeader** — the public/auth header with Log In / Sign Up.

10.4 Feature components (components/features)

The interactive pieces, grouped by domain: `auth` (LoginForm, RegisterForm, FormInput, LogoutButton), `pet` (AddPetForm, EditPetForm, PetPhotoUpload), `health-record` (AddHealthRecordForm), `appointment` (BookAppointmentForm, CancelButton, StatusActions), `clinic` (ClinicProfileForm) and `user` (ProfileForm). Each is a Client Component that collects input and calls a Server Action.

11 Feature Walkthroughs

Every feature follows the same shape. These walkthroughs trace the flow from page to action to API.

11.1 Authentication

The `(auth)` pages render a centred card. `LoginForm` validates with react-hook-form, calls `loginAction` (which runs next-auth `signIn`), and redirects to the role's dashboard. `RegisterForm` toggles between Owner and Clinic-Staff, shows the clinic dropdown for staff, and on success routes to /login.

11.2 Pets

`owner/pets` lists the owner's pets as interactive cards (or an empty state). `AddPetForm` / `EditPetForm` collect details with an optional date-picker birth date; `PetPhotoUpload` posts an image to Cloudinary via the backend. The detail page shows the pet header plus the portable health passport.

11.3 Health passport

The differentiator. An owner's pet detail page renders every `HealthRecord` across all clinics, each stamped with its source clinic and typed (vaccine vs. treatment). Owners see it read-only; clinic staff add records from `staff/pets/[petId]` via `AddHealthRecordForm`.

11.4 Clinics & booking

`owner/clinics` offers search/filter over the seeded clinics; the detail page shows info and a Google Maps link with a 'Book here' action that deep-links into the booking form with the clinic pre-selected. `BookAppointmentForm` collects pet + clinic + datetime + notes and calls `bookAppointmentAction`.

11.5 Appointments

Owners see their appointment list with statuses and a Cancel action on cancellable states. Staff see their clinic's queue split into new requests and the rest, with `StatusActions` driving the lifecycle (confirm / reject / complete / cancel) — each change writes a status log on the backend.

11.6 Profile

Both roles share `ProfileForm` (name + phone + avatar upload). Staff additionally edit their clinic via `ClinicProfileForm`. Profile updates call `unstable_update` so the session reflects the new name/avatar immediately.

12 Styling & the Design System

12.1 Tailwind v4 setup

Styling is Tailwind CSS v4, imported in `globals.css` alongside `tw-animate-css` and the shadcn token bridge. Class names are composed with the `cn()` helper (`clsx` + `tailwind-merge`) so conditional and overriding classes merge cleanly.

12.2 Design tokens & the .pw-* layer

The product's visual language — ivory canvas `#FAF8F4`, charcoal ink `#2D2A26`, dusty-blue brand `#5E86AD`, Inter + Playfair Display type — is defined in globals.css and applied through the `.pw-btn-primary`, `.pw-lift` and `.pw-fade` utilities plus the six-colour StatusBadge set. The complete token catalogue, every component's states, and usage rules live in the standalone `DESIGN_SYSTEM.html` reference page in this same docs folder.

> **One known nuance** The shadcn --primary token in globals.css is left at its greyscale default; the brand colour is applied via arbitrary values and inline styles rather than through that token. See the Design System page's 'Inconsistencies' section.

13 Cross-Cutting Concerns

13.1 Data fetching pattern

Server Components fetch in parallel with `Promise.all` and defensively `.catch(() => [])` so a failed call degrades to an empty state rather than a crash. This keeps pages resilient during the demo, at the cost of masking genuine API errors as 'empty'.

13.2 Loading & pending states

Writes use React's `useTransition`: while an action is pending the submit button disables and swaps its label to 'Saving…' / 'Requesting…', or shows an inline spinner. There is no skeleton system — feedback is via pending buttons and toasts.

13.3 Error handling & empty states

Form-level errors render as a danger-tinted banner; field errors as small red text under the input; action failures as a `toast.error`. Every list has a designed empty state with an icon, a line of copy and a primary action.

13.4 Images

Pet and clinic photos are user uploads stored on Cloudinary and rendered with plain `<img>` tags; the `PhotoPlaceholder` fills the gap when no image exists. (Uploads themselves are multipart FormData posts handled by the backend.)

13.5 Accessibility

Interactive elements use native buttons/links and visible focus rings (`focus:ring-2` in dusty blue). The profile menu sets `aria-haspopup`, `aria-expanded` and menu roles; icon-only buttons carry `aria-label`. Primary touch targets are 44px. Full notes are in the Design System page.

A Appendix A — Route Map

Route	Area	Renders
/	Public	Landing page
/login	(auth)	LoginForm
/register	(auth)	RegisterForm (owner / clinic-staff)
/owner/dashboard	Owner	Stats + upcoming appointments
/owner/pets	Owner	Pet list / empty state
/owner/pets/new	Owner	AddPetForm
/owner/pets/[petId]	Owner	Pet detail + health passport
/owner/pets/[petId]/edit	Owner	EditPetForm
/owner/clinics	Owner	Clinic search / filter
/owner/clinics/[clinicId]	Owner	Clinic detail + Book here
/owner/appointments	Owner	Appointment list + cancel
/owner/appointments/book	Owner	BookAppointmentForm
/owner/profile	Owner	ProfileForm
/staff/dashboard	Staff	Pending requests + schedule
/staff/appointments	Staff	Queue + StatusActions
/staff/pets/[petId]	Staff	Pet lookup + AddHealthRecordForm
/staff/clinic	Staff	ClinicProfileForm
/staff/profile	Staff	ProfileForm

B Appendix B — Server Actions Reference

Action	Calls	Revalidates
loginAction	signIn('credentials')	→ redirect '/'
registerAction / …ClinicStaff	AuthApi.register*	→ /login
logoutAction	signOut	→ /login
createPetAction	PetApi.create	/owner/pets, /owner/dashboard
updatePetAction	PetApi.update	/owner/pets/[id], /owner/pets

Action	Calls	Revalidates
uploadPetPhotoAction	PetApi.uploadPhoto	/owner/pets/[id], /owner/pets
archivePetAction	PetApi.archive	/owner/pets
bookAppointmentAction	AppointmentApi.create	/owner/appointments, /owner/dashboard
cancelAppointmentAction	AppointmentApi.cancel	/owner/appointments
updateAppointmentStatusAction	AppointmentApi.updateStatus	/staff/appointments, /staff/dashboard
createHealthRecordAction	HealthRecordApi.create	/owner/pets/[id], /staff/pets/[id]
updateClinicAction	ClinicApi.update	/staff/clinic
updateProfileAction	UserApi.updateProfile	/owner/profile, /staff/profile
uploadAvatarAction	UserApi.uploadAvatar	/owner/profile, /staff/profile

C Appendix C — API Client Reference

Client.method	HTTP → endpoint	Auth
AuthApi.login	POST /auth/login	public
AuthApi.register	POST /auth/register	public
AuthApi.registerClinicStaff	POST /auth/register/clinic-staff	public
AuthApi.me	GET /auth/me	token
PetApi.listMine	GET /pets	auth
PetApi.get	GET /pets/:id	auth
PetApi.create	POST /pets	auth
PetApi.update	PATCH /pets/:id	auth
PetApi.archive	DELETE /pets/:id	auth
PetApi.uploadPhoto	PATCH /pets/:id/photo	auth
ClinicApi.list	GET /clinics?search=&type=	public
ClinicApi.get	GET /clinics/:id	public
ClinicApi.update	PATCH /clinics/:id	auth
HealthRecordApi.listByPet	GET /pets/:id/health-records	auth
HealthRecordApi.get	GET /health-records/:id	auth
HealthRecordApi.create	POST /pets/:id/health-records	auth
HealthRecordApi.update	PATCH /health-records/:id	auth
HealthRecordApi.archive	DELETE /health-records/:id	auth

Client.method	HTTP → endpoint	Auth
HealthRecordApi.uploadAttachment	POST /health-records/:id/attachments	auth
AppointmentApi.listMine	GET /appointments	auth
AppointmentApi.create	POST /appointments	auth
AppointmentApi.cancel	DELETE /appointments/:id	auth
AppointmentApi.listByClinic	GET /clinics/:id/appointments	auth
AppointmentApi.updateStatus	PATCH /appointments/:id/status	auth
AppointmentApi.get	GET /appointments/:id	auth
AppointmentApi.logs	GET /appointments/:id/logs	auth
UserApi.updateProfile	PATCH /users/me	auth
UserApi.uploadAvatar	PATCH /users/me/avatar	auth
UserApi.uploadCover	PATCH /users/me/cover	auth

D Appendix D — Glossary

Term	Meaning
Server Component	React component rendered on the server; the default under app/. Can await data; no state.
Client Component	Component marked 'use client'; runs in the browser with hooks and events.
Server Action	'use server' function callable from the client; runs on the server for writes.
Route group	Folder in (parentheses) grouping routes without adding a URL segment, e.g. (auth).
authFetch	Data-layer helper that attaches the JWT and redirects to /login if unauthenticated.
ActionResult	Discriminated union every Server Action returns: success or a typed failure.
revalidatePath	Next API that marks a route's cached render stale so it re-renders.
requireRole	Session helper that enforces a role in a layout and redirects on mismatch.
cn()	Class-name helper: clsx + tailwind-merge, for composing Tailwind classes.

E Appendix E — Running the Frontend

Prerequisites

- Node.js and pnpm installed
- The PetWell backend running (default http://localhost:8000)

Environment (frontend/.env.local)

```
API_URL="http://localhost:8000"   # backend base URL
AUTH_SECRET="<random secret>"      # next-auth signing secret
```

Install & run

```
cd frontend
pnpm install
pnpm dev        # http://localhost:3000

pnpm build      # production build
pnpm start      # serve the build
```

Log in with a seeded account — for example the Pet Owner `owner@test.com` / `pass123` or Clinic Staff `vet@test.com` / `pass123` — to explore both areas.

— End of document —